PROJECT SHOWCASE

Building an Enhanced MCP Server & Client in Python

Bridging LLMs (Claude 3.7 Sonnet) with the Local File System safely and interactively using Anthropic's Model Context Protocol.

Python 3.11+

Asyncio, Pydantic V2

Model Context Protocol

Anthropic's FastMCP

Claude SDK

Interactive Agentic Loop

Swipe to see the tech ->

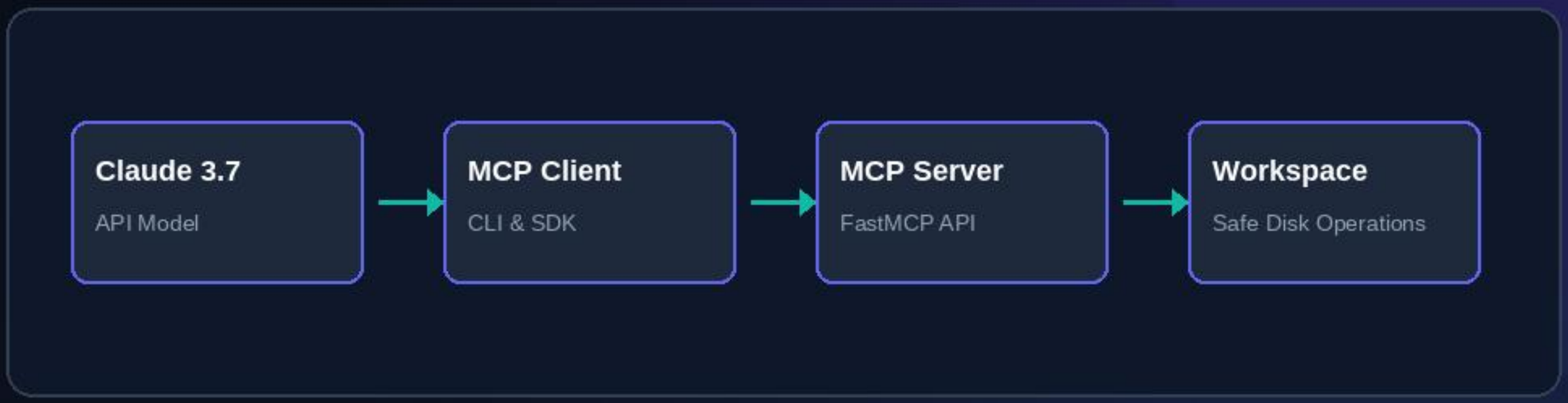
THE PROBLEM & THE SOLUTION

Why Model Context Protocol (MCP)?

- **The Challenge**: LLMs are isolated brains. They cannot directly read your workspace, run local commands, or update files without copy-pasting.
- **The Solution (MCP)**: An open standard created by Anthropic that allows clients (like Claude) to connect to secure server APIs over simple transports (like Stdio).
- **Key Value Proposition**:
- • **Security**: Server constrains operations inside the project root.
- • **Capability**: Lets LLMs write code, verify logs, and fetch project info.
- • **Decoupling**: Write server capabilities once, use across any Compliant Client.

SYSTEM ARCHITECTURE

Architecture: Client-Server Loop



- **Agentic Tool Loop**: When the LLM decides to call a tool (like `write_file`), the Client intercepts the request, calls the local MCP Server, and forwards the server's output back to Claude.
- **Context & Resources**: Resources (workspace structure, directory files) are dynamically mounted to the server for LLM consumption via custom protocols (`file://` and `dir://`).

FEATURE SHOWCASE #1

Real-Time Write Progress Reporting

For large writes or long operations, waiting blindly degrades UX. The server utilizes `ctx.report_progress` to update the client iteratively.

Server Implementation (Python)

```python
@mcp.tool()
async def write_file(file_path: str, content: str, ctx: Context) -> str:
    # Write file content in chunks with progress tracking
    for i in range(0, total, chunk_size):
        f.write(content[i:i+chunk_size])
        await ctx.report_progress(progress=written, total=total,
                                  message=f"Writing: {written}/{total}")
```

Client CLI Console Output

```
Connecting to server: server.py
Progress: 10.0% - Writing: 10/100
Progress: 50.0% - Writing: 50/100
Progress: 100.0% - Write complete
File written successfully to: sample.txt
```

FEATURE SHOWCASE #2

Dynamic Input Elicitation

Instead of hardcoding arguments, the server elicits structured input dynamically from the client. The client renders interactive prompts mapped directly to a Pydantic model.

Server Elicitation Implementation

```python
class DocumentGeneratorSchema(BaseModel):
    file_path: str
    name: str

@mcp.prompt()
async def documentation_generator(ctx: Context) -> str:
    # Ask the user for input matching the schema
    result = await ctx.elicit(
        message="Provide file names",
        response_type=DocumentGeneratorSchema
    )
    # result.data contains validated file_path and name!
```

Client Console Prompts

```
Select from the Menu > 1 (Generate Documentation)
Server asks: Provide file names
Enter value for 'file_path' (str): client.py
Enter value for 'name' (str): client_documentation.md
Generating prompt...
```

FEATURE SHOWCASE #3

Structured Resources & Prompts

Universal Resources

- **URI Schema mapping**:
  Exposes custom `file:///` and `dir://` URIs standard templates.
- **Rich Metadata**:
  Directory lists are returned with file size, modified timestamps, and node types.
- **Standard-Compliant**:
  Claude reads file contexts seamlessly.

Pre-Packaged Prompts

- **Interactive Workflows**:
  Bundled templates guide Claude in engineering tasks.
- **Code Review Prompt**:
  Automatically reads code file, sets strict criteria, and requests feedback.
- **Doc Generator Prompt**:
  Creates technical md docs naming them exactly as elicited.

- Both client and server utilize the standard model context protocol.
- The server registers endpoints dynamically, letting any compliant client (Cursor, Claude Desktop) use them out of the box.

CONCLUSION & CONTACT

Engineering Highlights & Let's Connect!

- **Modern Tech Stack**: Harnessing python `asyncio`, Pydantic V2 verification, and the official Anthropic SDK.
- **Standard-Driven Dev**: Implementing Model Context Protocol (MCP) spec v1.15.
- **Clean Engineering**: Sandboxing filesystem commands via resolved paths, preventing parent directory traversal hacks.

Looking for an internship or junior developer role!

I love building production-grade agentic frameworks, standardizing APIs, and working at the frontier of LLM integrations. Let's connect, share feedback, or discuss MCP in the comments! repo link is in my bio.